Daily Risk Pack

A Multi-Asset Risk Reporting System

SQL → Python → Excel, built on Linux

1 Objective and Constraints

1.1 What this project is for

This is a portfolio artifact and a training rig at the same time. It has two independent goals, and neither should be sacrificed for the other:

1. **Demonstrate risk competence.** Produce the kind of daily pack a middle-office or market-risk analyst ships every morning: exposures, VaR, limit utilisation, scenario results, and a backtest. The content should be recognisable to anyone who has worked a risk desk.
2. **Build Excel muscle memory.** The workbook is not a by-product. Rebuilding it repeatedly, keyboard-only, under a clock, is the mechanism by which spreadsheet speed is acquired. The project is designed so that this repetition has a purpose rather than feeling like drill for its own sake.

1.2 Hard constraints

- Linux host. No personal Windows machine. Excel runs in a KVM guest.
- Real desktop Excel only. LibreOffice teaches divergent shortcuts and formula behaviour and is excluded.
- Deliverables must contain *live formulas*, not pasted values. A workbook of hardcoded numbers defeats the entire purpose — the reviewer must be able to change an assumption and watch it propagate.

1.3 Positioning

The interview framing is: fast and clean in Excel, *and* able to automate the data pipeline feeding it. That is a strict upgrade over the median candidate. Python is presented as additive, never as a replacement. The phrase "I prefer Python to spreadsheets" should not be spoken in a first-round interview.

2 Environment

2.1 Virtualisation stack

Component	Choice	Notes
Hypervisor	KVM/QEMU + libvirt	`virt-manager` for the GUI
Guest	Windows 11	4 vCPU, 8 GB RAM, 60 GB disk
Office	Office Home 2024	Perpetual, one-time cost
Display	SPICE, `virt-viewer`	Fullscreen, keyboard grabbed
File exchange	virtiofs or Samba	Host dir mounts as `Z:` in guest
Database	DuckDB (host)	Postgres if you want ODBC realism

2.2 The keyboard problem

This is the step that quietly ruins the whole exercise if skipped. The `Alt` chord tree *is* Excel's command language, and most Linux window managers intercept exactly those keys.

- ☐ Rebind host WM modifier to `Super` only; strip all `Alt` bindings on the VM workspace.
- ☐ Launch with `virt-viewer --attach --full-screen`; set the keyboard-release hotkey to something you would never press by accident.
- ☐ Verify `Ctrl+Alt+F1`–`F12` are not bound to VT switching.
- ☐ **Acceptance test:** open Excel, press `Alt` alone. If ribbon key tips appear, the environment is clean.

2.3 Excluded approaches

Wine/CrossOver with modern Office is unreliable. Excel for the web drops enough shortcuts to be actively misleading. Cloud Windows works but adds input latency, which is worst precisely for shortcut drilling.

3 System Architecture

Layer	Technology	Responsibility
Ingest	`yfinance`, FRED API, Python	Fetch prices, FX, rates; write raw tables
Store	DuckDB (or Postgres)	Normalised schema, referential integrity
Transform	SQL	Joins, returns, aggregation, data-quality checks
Analytics	Python (`numpy`, `pandas`)	VaR, ES, component VaR, scenarios, backtest
Emit	`xlsxwriter` / `openpyxl`	Workbook with live formulas and named ranges
Present	Excel (guest VM)	Power Query, pivots, data tables, Solver

The division of labour is deliberate. Anything a reviewer might want to interrogate — weights, matrix products, scenario shocks, limit tests — lives in Excel as a formula. Anything that is pure plumbing — API calls, covariance estimation, path generation — lives upstream.

4 Data Layer

4.1 Schema

Table	Columns
`instruments`	`id`, `ticker`, `asset_class`, `sector`, `country`, `ccy`
`positions`	`as_of_date`, `instrument_id`, `quantity`
`prices`	`as_of_date`, `instrument_id`, `close`, `ccy`
`fx_rates`	`as_of_date`, `ccy`, `usd_rate`
`limits`	`scope`, `scope_value`, `metric`, `limit_value`

4.2 Synthetic book design

Construct 40–60 names with deliberate awkwardness, because clean data teaches nothing:

- Several short positions (negative quantity), so sign handling is tested.

- At least three non-USD positions, so the FX join is load-bearing.
- Two instruments with a stale price run (identical closes for a week).
- One instrument in `positions` with no matching `instruments` row, to be caught by the orphan check.
- Mixed asset classes: equities, ETFs, a rates proxy, a commodity proxy.

5 SQL Layer

Aggregation happens in SQL, not pandas. This is a rule, not a preference — SQL is the highest-leverage skill to demonstrate for these roles.

Query	Technique exercised
Daily returns	`LAG(close) OVER (PARTITION BY instrument_id ORDER BY as_of_date)`
Market value	Three-way join: positions → prices → fx, with sign handling
Exposure pivot	`GROUPING SETS` / `ROLLUP` by sector and country
Price carry-forward	As-of join for closed-market dates
Orphan positions	Anti-join against `instruments`
Missing prices	Calendar cross-join, left join, null filter
Stale price detection	Window function counting consecutive identical closes

6 Analytics

6.1 Historical simulation

Revalue the current book under each of the last 250 daily return vectors. VaR at confidence α is the empirical quantile of the resulting P&L distribution; expected shortfall is the mean of the tail beyond it:

$$\mathrm{ES}_\alpha = \mathbb{E}[\, L \mid L > \mathrm{VaR}_\alpha \,]$$

6.2 Parametric VaR with EWMA volatility

Exponentially weighted variance, RiskMetrics convention:

$$\sigma_t^2 = \lambda \sigma_{t-1}^2 + (1-\lambda) r_{t-1}^2, \qquad \lambda = 0.94$$

Portfolio VaR for value V and weight vector $\mathbf{w}$:

$$\mathrm{VaR}_\alpha = z_\alpha \sqrt{\mathbf{w}^\top \Sigma \mathbf{w}} \cdot V$$

6.3 Component VaR

The decomposition risk committees actually read, since it identifies *who* is driving the number:

$$\frac{\partial \mathrm{VaR}}{\partial \mathbf{w}} = z_\alpha \frac{\Sigma \mathbf{w}}{\sqrt{\mathbf{w}^\top \Sigma \mathbf{w}}}, \qquad \mathrm{CVaR}_i = w_i \frac{\partial \mathrm{VaR}}{\partial w_i}$$

Euler's theorem gives a free correctness check:

$$\sum_i \mathrm{CVaR}_i = \mathrm{VaR}_\alpha$$

6.4 Monte Carlo VaR

Cholesky-factor the covariance matrix, $\Sigma = LL^\top$, and map independent standard normals to correlated draws via $\mathbf{x} = L\mathbf{z}$. Price paths under geometric Brownian motion:

$$S_{t+\Delta t} = S_t \exp\left[\left(\mu - \tfrac{1}{2}\sigma^2\right)\Delta t + \sigma\sqrt{\Delta t}\, Z\right]$$

Convergence is the honest constraint — standard error falls as

$$\text{SE} = \frac{\sigma}{\sqrt{N}}$$

so each additional digit of precision costs $100\times$ the paths. Excel is comfortable at $N = 10^4$, slow at 10^5, and unusable at 10^6. Antithetic variates ($1 - U$ paired with U) are nearly free and cut variance substantially for monotone payoffs.

6.5 Backtest

Count VaR exceptions and test whether the failure rate matches the nominal level. Kupiec proportion-of-failures statistic, with x exceptions in N observations at nominal rate p and $\hat{p} = x/N$:

$$LR_{POF} = -2\ln\left[\frac{(1-p)^{N-x}\,p^x}{(1-\hat{p})^{N-x}\,\hat{p}^{\,x}}\right] \sim \chi^2_1$$

6.6 Stress scenarios

Replay historical windows against the current book: September–October 2008, February–March 2020, and the 2022 rate selloff. Report P&L, not returns — the audience thinks in dollars.

7 The Excel Deliverable

7.1 Workbook structure

Tab	Contents
`Summary`	Headline VaR/ES, limit utilisation with RAG conditional formatting, day-over-day delta
`Positions`	Position detail, market value, FX-adjusted exposure
`Risk`	Covariance block, `MMULT` chain, component VaR table
`Scenarios`	Two-way data table: equity shock × rate shock
`Backtest`	Exception timeline, Kupiec statistic, traffic-light zone
`Calc`	Hidden intermediate workings

7.2 Formatting conventions

Non-negotiable, because they are graded in modelling tests:

- Blue font for inputs, black for formulas, green for cross-sheet links.
- No hardcoded constants buried inside formulas.
- One consistent formula copied across each row — no exceptions mid-range.
- Named ranges for every input block and every output referenced elsewhere.

7.3 Excel features exercised

Feature	Where it appears
Power Query	ODBC connection to DuckDB/Postgres. *The single most valuable item on this list* — it is the literal bridge between your toolchain and theirs.
`MMULT`, `TRANSPOSE`	Computing $\mathbf{w}^\top \Sigma \mathbf{w}$; forces correct dynamic-array spilling
Two-way data table	Scenario grid; also the native Monte Carlo loop
Solver	Minimum-variance overlay or optimal hedge ratio
`LET`, `LAMBDA`	Named sub-expressions in the risk calculations
`FILTER`, `SORT`, `UNIQUE`	Dynamic position and exception tables
`SUMIFS`, `XLOOKUP`	Exposure rollups, instrument attribute lookup
PivotTables + slicers	Exposure by sector/country/asset class
Conditional formatting	Limit RAG status, backtest exception flags
Sparklines	Inline VaR and exposure history

7.4 Native Monte Carlo without VBA

Worth knowing as a standalone trick. Build the model so one cell holds the result. Put trial indices $1 \dots N$ down a column. Point a one-variable data table's *column input cell* at any empty cell the model does not reference. Excel recalculates the entire sheet once per row, and because `RAND()` is volatile, each recalculation redraws. Set calculation to manual while building, and freeze the final run with a values-only paste (`Ctrl+Shift+V`) before presenting — otherwise the numbers change while the reviewer is looking at them.

8 Phased Roadmap

Phase 0 — Environment

Deliverable

A Windows guest running Excel, keyboard fully grabbed, with a shared folder mounted as a drive letter.

☐ VM provisioned and Office licensed.

☐ Host WM `Alt` bindings stripped on the VM workspace.

☐ Ribbon key tips confirmed working.

☐ virtiofs share writable from host, readable from guest.

☐ Mouse physically unplugged.

Done when: you can open a workbook written by a host-side Python script without touching a pointing device.

Phase 1 — Data and SQL

Deliverable

A populated database and a file of named, commented SQL queries producing exposures and returns.

☐ Schema created; synthetic book loaded with the deliberate defects.

☐ Price and FX history ingested for ≥ 3 years.

☐ Returns, market value, and exposure queries written and verified.

☐ All four data-quality checks return the planted defects and nothing else.

Done when: a single command rebuilds the database from scratch and every quality check passes or flags exactly what it should.

Phase 2 — Analytics

Deliverable

A Python module computing all three VaR methodologies plus ES, component VaR, scenarios, and the backtest.

☐ Historical simulation VaR and ES at 99%/1d.

☐ EWMA covariance; parametric VaR.

☐ Component VaR, validated against $\sum_i \text{CVaR}_i = \text{VaR}$.

☐ Monte Carlo VaR with Cholesky-correlated draws.

☐ Kupiec test over a rolling backtest window.

☐ Three historical stress windows replayed.

Done when: you can explain, out loud and unprompted, why the three VaR numbers disagree.

Phase 3 — Workbook generation

Deliverable

`risk_pack_YYYYMMDD.xlsx`, generated end-to-end from one command, containing live formulas throughout.

☐ `xlsxwriter` emits formulas, not values — verified by changing an input and watching the summary move.

☐ Named ranges defined programmatically.

☐ Formatting conventions applied (blue/black/green).

☐ Conditional formatting on limit utilisation.

☐ Power Query connection established as a second, independent path to the same data.

Done when: an MD could open it cold, change one assumption, and trace the effect without asking you anything.

Phase 4 — Manual rebuild and drilling

Deliverable

A timing log showing the `Risk` tab rebuild time falling week over week.

☐ Build v0 entirely by hand, mouse unplugged, untimed.

☐ Delete the `Risk` tab; rebuild from scratch, timed.

☐ Repeat weekly, logging the time.

☐ Maintain a scratch file of every mouse-reach instinct; look up the `Alt` chord the same session and drill it ten times.

Done when: rebuild time plateaus and the mouse-reach log stops gaining entries.

Phase 5 — Risk attribution (the differentiator)

Deliverable

A day-over-day decomposition of the change in VaR into position, price, and volatility effects.

Nobody asks "what is our VaR." They ask "why did VaR jump 12% overnight." Being able to answer that, with the decomposition on a page, is the job. This is the section that makes an interviewer stop skimming.

☐ Hold two of {positions, prices, volatilities} fixed; vary the third.

☐ Residual term reported explicitly rather than hidden.

☐ Waterfall chart on the `Summary` tab.

9 Practice Protocol

Cadence	Activity
Daily (15 min)	Ten `Alt` chords from the mouse-reach log, cold
Weekly	Timed rebuild of one workbook tab from scratch
Weekly	Regenerate the pack against fresh market data
Monthly	Full modelling-test simulation: blank workbook, 90-minute clock

9.1 Core navigation to internalise first

Key	Action
`Alt`	Opens the ribbon chord tree — the whole command language
`Ctrl+arrow`	Word-motion to region edge
`Ctrl+Shift+arrow`	Visual-mode word-motion
`Ctrl+Space` / `Shift+Space`	Select column / row
`F5` + name	Jump to named range (tag-jump)
`Ctrl+[`	Jump to formula precedents (go-to-definition)
`F5 Enter`	Jump back (pop-tag)
`F2`	Edit in place
`F4`	Cycle absolute/relative references
`Alt+=`	Autosum
`Ctrl+PgUp/PgDn`	Sheet switching

10 Stretch Goals

- **Liquidity-adjusted VaR.** Scale the horizon by an instrument-specific unwind period; report the uplift separately.
- **Marginal contribution to limit breach.** Which single position, if halved, brings the book back inside limits?
- **Incremental VaR for a proposed trade.** Pre-trade check — exactly what a risk desk fields all day.
- **Distribution fitting.** Replace the normal assumption with a Student-t; show the tail difference and quantify the parametric understatement.
- **PDF pack generation.** The workbook is the working artifact; a two-page PDF is what actually circulates.

11 Interview Framing

Three things to be able to say without hesitation:

1. **Why the three VaR numbers disagree.** Parametric understates tails by construction. Historical simulation is hostage to whatever the last 250 days happened to contain. Monte Carlo is only as good as its distributional assumption. Choosing between them is a judgement call, and having the judgement is the point.

2. **Why the workbook contains formulas rather than values.** Because the deliverable has to be legible and interrogable by the reviewer, not optimal for the author. A spreadsheet is an auditable dataflow graph; a notebook is opaque to the person who needs to sign off.

3. **Where automation ends and Excel begins.** You automate the plumbing and leave the judgement layer visible. That is the correct division, and articulating it is what separates "the Python person" from "the person who makes the team faster."